



Trailhead



[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Migrate Markup and CSS

Migrate Markup and CSS

Learning Objectives

After completing this unit, you'll be able to:

- Migrate HTML and markup from Aura components to Lightning web components.
- Migrate CSS from Aura components to Lightning web components.

The DreamHouse App

The best way to compare the Aura Component and Lightning Web Components programming models is to look at code for similar components written in the two models.

We're going to look at some components that are part of the DreamHouse app, which is a relatively complex app that Salesforce has developed to showcase Lightning components. DreamHouse is a sample application for a real-estate business built on the Salesforce platform. The app enables brokers to manage their properties, and enables buyers to find their dream home.

The DreamHouse app has separate implementations for Aura components and Lightning web components, and the full code for each is available in two different GitHub repos. The functionality isn't identical in both versions of the app, but there are a few components that are useful for comparing the two programming models.

The code for the two implementations of the DreamHouse app is available in GitHub repos.

- [DreamHouse Aura component GitHub repo](https://github.com/trailheadapps/dreamhouse-sfdx)
(<https://github.com/trailheadapps/dreamhouse-sfdx>)
- [DreamHouse Lightning web components GitHub repo](https://github.com/trailheadapps/dreamhouse-lwc)
(<https://github.com/trailheadapps/dreamhouse-lwc>)





Component Bundles

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

The component bundle file structure is different for an Aura component and a Lightning web component. Here's how the files map between the two types of component.

Migrate Markup and CSS

Resource	Aura File	Lightning Web Components File
Markup	sample.cmp	sample.html
Controller	sampleController.js	sample.js
Helper	sampleHelper.js	
Renderer	sampleRenderer.js	
CSS	sample.css	sample.css
Documentation	sample.auradoc	Not currently available
Design	sample.design	sample.js-meta.xml
SVG	sample.svg	Include in HTML file or upload as a static resource



The separate controller, helper, and renderer files in an Aura component map to one JavaScript file in a Lightning web component.



In this unit, we look at the HTML and CSS files. In the next unit, we explore the JavaScript file.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Lightning web components also have a configuration file, which defines the metadata values for the component, including the design configuration for components intended for use in Lightning App Builder. The configuration file houses some of the metadata that is defined by interfaces, such as `flexipage:availableForRecordHome`, in Aura components.

Migrate Markup

An Aura component contains markup in a `.cmp` file. The file starts with an `<aura:component>` tag and can contain HTML and Aura-specific tags.

A Lightning web component contains markup in a `.html` file. The file starts with a `<template>` tag and can contain HTML and directives for dynamic content.

Let's see how the Aura markup maps to content in the HTML and JavaScript files of a Lightning web component.

Attributes Become JavaScript Properties

Migrate attributes from `<aura:attribute>` tags in the markup (`.cmp`) of an Aura component to JavaScript properties in the JavaScript file (`.js`) of a Lightning web component.

Let's look at these attributes in the markup for the PropertySummary Aura component. You can see the complete file in the [DreamHouse Aura component GitHub repo \(https://github.com/trailheadapps/dreamhouse-sfdx\)](https://github.com/trailheadapps/dreamhouse-sfdx).

```
1 <aura:attribute name="recordId" type="Id" />
2   <aura:attribute name="property" type="Property__c" />
```



In a Lightning web component, we use JavaScript properties instead of attributes in markup. Let's look at the properties in `propertySummary.js`.

```
1 import { LightningElement, api } from 'lwc';
2 export default class PropertySummary extends LightningElement {
3   @api recordId;
```





The recordId and property attributes in the Aura component become the recordId and property JavaScript properties in the Lightning web component.

Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
The @api decorator marks recordId as a public property. A public property is part of the public API for the component, which means that it can be set in Lightning App Builder, or by a parent component that uses the component in its markup.
Migrate Markup and CSS

Basic Aura Expressions Become HTML Expressions

Migrate basic expressions from markup in an Aura component to expressions in HTML in a Lightning web component. An example of a basic expression is a reference to an attribute in an Aura component.

For example, the PropertyPaginator Aura component uses basic expressions to display the values of the page, pages, and total attributes.

```
1 <aura:component >
2   <aura:attribute name="page" type="integer"/>
3   <aura:attribute name="pages" type="integer"/>
4   <aura:attribute name="total" type="integer"/>
5   <div class="centered">{!v.total} properties • page {!v.page}
6   of {!v.pages}</div>
  </aura:component>
```



Here's the equivalent syntax in the HTML file of the paginator Lightning web component.

```
1 <template>
2   {totalItemCount} items • page {currentPageNumber} of
3   {totalPages}
  </template>
```



Dynamic content in a Lightning web component's HTML file doesn't have an exclamation point or value provider (v.) syntax. Don't use the expression syntax from Aura components in a Lightning web component even though your fingers might be used to typing it!



```

1  import { LightningElement, api } from 'lwc';
2  export default class Paginator extends LightningElement {
3      /** The current page number. */
4      @api pageNumber;
5      /** The number of items on a page. */
6      @api pageSize;
7      /** The total number of items in the list. */
8      @api totalItemCount;
9      get currentPageNumber() {
10         return this.totalItemCount === 0 ? 0 : this.pageNumber;
11     }
12     get totalPages() {
13         return Math.ceil(this.totalItemCount / this.pageSize);
14     }
15 }

```

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Migrate Markup and CSS

We tackle more complex expressions soon after we understand conditional statements.

Aura Conditionals Become HTML Conditionals

Migrate `<aura:if>` tags in an Aura component to `lwc:if`, `lwc:elseif`, or `lwc:else` directives in a Lightning web component's HTML file.

Here's some conditional markup in the BrokerDetails Aura component.

```

1  <aura:if.isTrue="{!v.property.Broker__c}">
2      <lightning:recordForm recordId="{!v.property.Broker__c}"
3          objectApiName="Broker__c"
4          fields="{!v.brokerFields}" columns="2"/>
5  </aura:if>

```

Here's similar HTML in the brokerCard Lightning web component.

```

1  <template lwc:if={property.data}>
2      <lightning-record-form object-api-name="Broker__c"
3          record-id={brokerId} fields={brokerFields}
4          columns="2">
5      </lightning-record-form>
6  </template>

```



The HTML file of a Lightning web component starts with the standard HTML `<template>` tag, and it can also contain other `<template>` tags in its body. In this example, the content in the `<template>` tag conditionally renders depending on the result of the `lwc:if` directive.

[Lightning Web Components for Aura Developers \ / content / learn / modules / lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Migrate Markup and CSS

Complex Aura Expressions Become JavaScript Logic

We saw how basic Aura expressions, such as references to attribute values, become HTML expressions. Migrate a complex Aura expression, such as a comparison operation or a ternary operator, in an Aura component to a JavaScript getter in a Lightning web component.

Aura components support a rich set of expressions, such as this example in the PropertyPaginator Aura component, which uses `{!v.page > 1}`.

```
1 <aura:if.isTrue="{!v.page > 1}">
2     <lightning:buttonIcon iconName="utility:left"
3     variant="border" onclick="{!c.previousPage}" />
</aura:if>
```



The expression language in Aura allows you to include more logic in your markup. The downside is that it's harder to unit test this logic, and the functionality is only a subset of what you can do in JavaScript.

In a Lightning web component, move the complex expression into JavaScript. Now the code can be unit tested, which makes for more stable code and happier developers. You also have access to the full capabilities of JavaScript to write your logic.

Here's similar HTML in the paginator Lightning web component.

```
1 <template lwc:if={isNotFirstPage}>
2     <lightning-button-icon icon-name="utility:chevronleft"
3     onclick={previousHandler}></lightning-button-icon>
</template>
```



The getter in paginator.js evaluates the `{isNotFirstPage}` condition. For example:

```
1 import { LightningElement, api } from 'lwc';
2 export default class Paginator extends LightningElement {
```





```
8 | }
```

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebeho7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebeho7&trailmix_slug=salesforce-platform-developer)

Aura Iterations Become HTML Iterations

Migrate Markup and CSS `for:each` tags in an Aura component to `for:each` directives in a Lightning web component's HTML file.

Here's the Aura syntax in `PropertyTileList.cmp`.

```
1 | <aura:iteration items="{!v.properties}" var="property">
2 |     <c:PropertyTile property="{#property}" />
3 | </aura:iteration>
```



Here's similar HTML in the `propertyTileList` Lightning web component.

```
1 | <template for:each={properties.data.records}
2 | for:item="property">
3 |     <c-property-tile property={property} key={property.Id}
4 |     onselected={handlePropertySelected}></c-property-tile>
   | </template>
```



Aura Initializers Become Lifecycle Hooks

Replace an `init` event handler in an Aura component with the standard JavaScript `connectedCallback()` method in a Lightning web component. The `connectedCallback()` lifecycle hook fires when a component is inserted into the DOM. Lifecycle hooks are callback methods that let you run code at each stage of a component's lifecycle.

We use the `init` event in an Aura component to initialize a component after component construction but before rendering.

Here's an `init` event handler in the `PropertyCarousel` Aura component.

```
1 | <aura:handler name="init" value="{!this}" action="{!c.onInit}"
   | />
```





```
1 export default class PropertySummary extends LightningElement {  
2     // ...  
3     connectedCallback() {  
4         // initialize component  
5     }  
6 }
```



Migrate Markup and CSS

Migrate Base Components

Remember how a Lightning web component can't contain Aura components? What happens if the Aura component you need to migrate contains built-in components that Salesforce supplies? Base Lightning components are building blocks that Salesforce provides in the lightning namespace, and they're available in both Aura components and Lightning web components. Base Lightning components have a different syntax when you use them in the two programming models.

This Aura component uses the lightning:formattedNumber base component.

```
1 <aura:component>  
2     <lightning:formattedNumber value="5000" style="currency"  
3     currencyCode="USD" />  
4 </aura:component>
```



To migrate this markup to a Lightning web component:

- Change the colon that separates the namespace (lightning) and component name (formattedNumber) to a dash.
- Change the camel-case component name (formattedNumber) to a dash-separated name (formatted-number).
- Change the camel-case attribute name (currencyCode) to a dash-separated name (currency-code).

Here's the equivalent Lightning web component.

```
1 <template>  
2     <lightning-formatted-number value="5000" style="currency"  
3     currency-code="USD">  
4
```





closing. Self-closing tags end with `/>`. A Lightning web component is essentially a custom element. Therefore, the `<lightning-formatted-number>` component reference includes a separate closing `</lightning-formatted-number>` tag. This requirement is different than Aura components, where self-closing tags are allowed.

Migrate Markup and CSS

Aura CSS Becomes Standard CSS

Lightning web components use standard CSS syntax. Remove the proprietary `THIS` class that Aura components use.

Here's a portion of the CSS file for the `PropertyTile` Aura component.

```
1  .THIS .lower-third {
2      position: absolute;
3      bottom: 0;
4      left: 0;
5      right: 0;
6      color: #FFF;
7      background-color: rgba(0, 0, 0, .3);
8      padding: 6px 8px;
9  }
10 .THIS .lower-third > p {
11     padding: 0;
12     margin: 0;
13 }
14 .THIS .truncate {
15     width: 100%;
16     white-space: nowrap;
17     overflow: hidden;
18     text-overflow: ellipsis;
19 }
```



Similar CSS for the `propertyTile` Lightning web component uses standard CSS instead. The `THIS` keyword is specific to Aura and isn't used in Lightning web components.

```
1  .lower-third {
2      position: absolute;
3  }
```





```
9 padding: 6px 8px;  
10 }
```

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers/migrate-markup-and-css?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers/migrate-markup-and-css?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

```
13 margin: 0;
```

Migrate Markup and CSS

```
15 .truncate {  
16     width: 100%;  
17     white-space: nowrap;  
18     overflow: hidden;  
19     text-overflow: ellipsis;  
    }
```

Style Components with Lightning Design System

Custom components that live in Lightning Experience or in the Salesforce mobile application can use Lightning Design System without any import statements or static resources. Assign a Lightning Design System CSS class to an HTML element, and you're good to go.

Lightning Web Components Inherit Aura Component Styling

Aura component styles cascade to any Lightning web components that they contain.

However, the styles of a Lightning web component never interfere with other Lightning web components or with Aura components. Yes, you read that correctly. Lightning web component styles don't cascade to their children. Overriding styles that you don't own can create many problems, so Lightning web components don't allow it.

CSS Encapsulation with Shadow DOM

Lightning web components use a web-standard mechanism called shadow DOM that hides the elements inside the component from the page that contains the component. Because Lightning web components have a shadow DOM, styles defined in a component's style sheet are scoped to the component. They don't apply to parent, child, or sibling components. This rule is strict, but it allows a component to be reused in different contexts without losing its styling. It also prevents a component's styles from overriding styles in other parts of a page.



Resources

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

- [Lightning Aura Components Developer Guide: Creating Components](#)

[Migrate Markup and CSS \(/content/learn/modules/lightning-web-components-for-aura-developers/migrate-markup-and-css?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

- [Lightning Aura Components Developer Guide: Configure Components for Lightning Pages and the Lightning App Builder \(/content/learn/modules/lightning-web-components-for-aura-developers/migrate-markup-and-css?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)
- [Lightning Aura Components Developer Guide: Configure Components for Lightning Pages and the Lightning App Builder \(https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/components_overview.htm\)](#)
- [Lightning Web Components Developer Guide: Component Lifecycle Hooks \(https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/components_config_for_app_builder.htm\)](#)
- [Lightning Web Components Developer Guide: Component Lifecycle Hooks \(https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.create_lifecycle_hooks\)](#)
- [Lightning Web Components Developer Guide: Component Configuration File \(https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.create_components_meta_file\)](#)

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points

1 What does an Aura attribute migrate to in a Lightning web component?

- A** A <template> tag
- B** A JavaScript function
- C** A JavaScript property



2 What styles does a Lightning web component's style sheet affect?

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Migrate Markup and CSS

A

The Lightning web component only

B

The Lightning web component and its child Lightning web components

C

The Lightning web component and its child Aura components

D

The Lightning web component and its parent Aura component

Second attempt earns 50 points. Three or more earns 25 points.

Check the Quiz to Earn 100 Points

Learn

Badges

Trails

Trailmixes

Superbadges

Trailhead Academy

Career Paths

Certifications

Certifications

Maintain Certifications

Verify Certifications

Community

Trailblazer Community

Events

Agentblazers

Salesforce Developers

Salesforce Admins

Code of Conduct

Report Illegal Content
(EU)

Extras

Trailhead Login

Sales Enablement

Trailblazer Store

Salesforce Help

Download Trailhead GO



[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)

 Engli 

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Migrate Markup and CSS